Beginner-SQL PLP Assignment

Q1) Topic: SELECT, WHERE, ORDER BY, LIMIT, Aliases & NULL Handling

Using a products table (id, name, category, price, stock_qty, supplier_id -- supplier_id may be NULL):

Query | Query History

```
CREATE TABLE products(
id INT PRIMARY KEY,
name VARCHAR(50),
category VARCHAR(50),
price DECIMAL(10,2),
stock_qty INT,
supplier_id INT
);
```

Data Output | Messages | Notifications

```
CREATE TABLE

Query returned successfully in 237 msec.
```

Added sample data:

Query | Query History

```
INSERT INTO products (id, name, category, price, stock_qty, supplier_id)
VALUES
    (1,  'Phone',          'Electronics', 999.00,  15, 101),
    (2,  'Laptop',         'Electronics', 1500.00, 10, 102),
    (3,  'Tablet',         'Electronics', 500.00,   0, NULL),
    (4,  'Headphones',     'Electronics', 750.00,  25, 103),
    (5,  'Smartwatch',     'Electronics', 450.00,   8, NULL),
    (6,  'Camera',         'Electronics', 1200.00,  5, 104),
    (7,  'Office Chair',   'Furniture',   650.00,  12, 105),
    (8,  'Desk',           'Furniture',   850.00,   0, NULL),
    (9,  'Notebook',       'Stationery',    50.00, 100, 106),
    (10, 'Pen Set',        'Stationery',   120.00,  40, NULL),
    (11, 'Monitor',        'Electronics',  550.00,  20, 102),
    (12, 'Keyboard',       'Electronics',  300.00,   0, 103);

```

Data Output | Messages | Notifications

```
INSERT 0 12

Query returned successfully in 85 msec.
```

a) List all 'Electronics' products costing more than 500, ordered by price descending. Alias name as product_name, price as unit_price.

Query | Query History

```sql
SELECT name as product_name,price as unit_price
FROM products
WHERE category='Electronics' AND price>500
ORDER BY unit_price DESC;
```

Data Output | Messages | Notifications

Showing rows: 1 to 5 | Page No: 1 of 1

	product_name character varying (50)	unit_price numeric (10,2)
1	Laptop	1500.00
2	Camera	1200.00
3	Phone	999.00
4	Headphones	750.00
5	Monitor	550.00

b) Find the 5 most expensive products overall.

Query | Query History

```sql
SELECT DISTINCT name,category,price FROM Products
ORDER BY price DESC
LIMIT 5;
```

Data Output | Messages | Notifications

Showing rows: 1 to 5 | Page No: 1 of 1

	name character varying (50)	category character varying (50)	price numeric (10,2)
1	Laptop	Electronics	1500.00
2	Camera	Electronics	1200.00
3	Phone	Electronics	999.00
4	Desk	Furniture	850.00
5	Headphones	Electronics	750.00

c) Use COALESCE to show supplier_id if available, otherwise 'No Supplier'. Use NULLIF to return NULL if stock_qty = 0.

Query | Query History

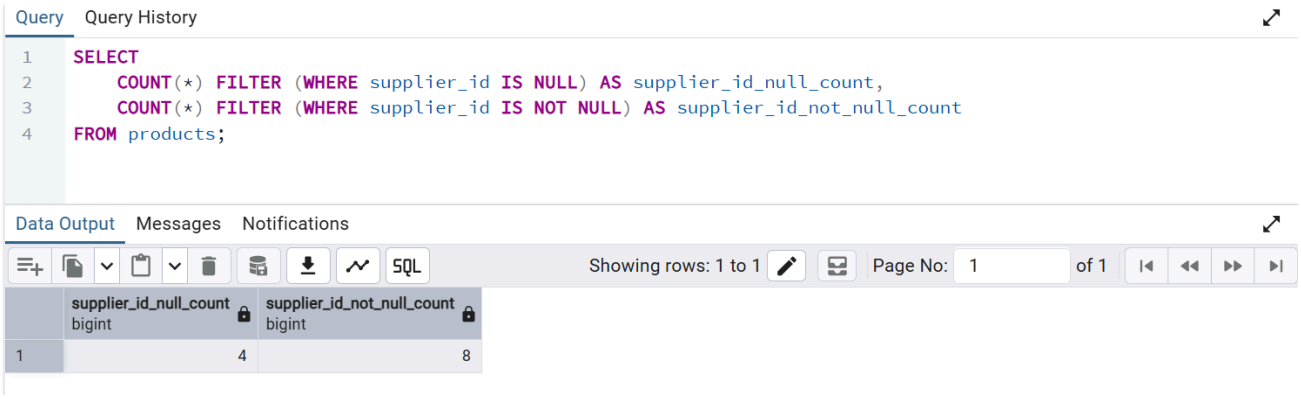
```sql
SELECT name,price,COALESCE(supplier_id::text,'No Supplier') as supplier_id, NULLIF(stock_qty,0) as stock_qty
FROM products;
```

Data Output | Messages | Notifications

Showing rows: 1 to 12 | Page No: 1 of 1

	name character varying (50)	price numeric (10,2)	supplier_id text	stock_qty integer
1	Phone	999.00	101	15
2	Laptop	1500.00	102	10
3	Tablet	500.00	No Supplier	[null]
4	Headphones	750.00	103	25
5	Smartwatch	450.00	No Supplier	8
6	Camera	1200.00	104	5
7	Office Chair	650.00	105	12
8	Desk	850.00	No Supplier	[null]
9	Notebook	50.00	106	100
10	Pen Set	120.00	No Supplier	40
11	Monitor	550.00	102	20
12	Keyboard	300.00	103	[null]

d) Find products where supplier_id IS NULL and where IS NOT NULL. Show counts using COUNT().



e) Predict Output: Predict the result set: -- products: (1,'Phone',999,NULL),(2,'Laptop',1500,10), (3,'Tablet',500,NULL)

```
SELECT name, COALESCE(supplier_id::text,'No Supplier') AS supplier,
NULLIF(price,500) AS price
FROM products
ORDER BY price DESC NULLS LAST;
```

Final result set:

name	supplier	price
Laptop	10	1500
Phone	No Supplier	999
Tablet	No Supplier	`NULL`

Q2) Topic: Aggregates: COUNT, SUM, AVG + GROUP BY & HAVING

Using orders (id, customer_id, order_date, total_amount) and customers (id, name, city):

Query | Query History

```sql
CREATE TABLE customers (
    id INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    city VARCHAR(100)
);
CREATE TABLE orders (
    id INT PRIMARY KEY,
    customer_id INT REFERENCES customers(id),
    order_date DATE,
    total_amount NUMERIC(10, 2)
);

```

Data Output | Messages | Notifications

```
CREATE TABLE

Query returned successfully in 245 msec.
```

Query | Query History

```sql
INSERT INTO customers (id, name, city)
VALUES
    (1, 'Aarav',  'Mumbai'),
    (2, 'Diya',   'Mumbai'),
    (3, 'Ishaan', 'Delhi'),
    (4, 'Meera',  'Delhi'),
    (5, 'Rohan',  'Bengaluru'),
    (6, 'Sara',   'Bengaluru'),
    (7, 'Kabir',  'Chennai');

INSERT INTO orders (id, customer_id, order_date, total_amount)
VALUES
    (1,  1, '2024-01-10', 4000),
    (2,  1, '2024-02-15', 3500),
    (3,  1, '2024-03-20', 3000),
    (4,  2, '2024-01-25', 2500),
    (5,  2, '2024-04-12', 1800),
    (6,  3, '2024-01-05', 6000),
```

Data Output | Messages | Notifications

```
INSERT 0 16

Query returned successfully in 68 msec.
```

a) Show city, number of customers, and total revenue. Only include cities with revenue > 10000. Sort by revenue descending.

Query | Query History

```sql
SELECT c.city, COUNT(DISTINCT c.id) AS num_of_customers, SUM(o.total_amount) AS total_revenue
FROM customers c
JOIN orders o
ON c.id=o.customer_id
GROUP BY c.city
HAVING SUM(o.total_amount)>10000
ORDER BY total_revenue DESC;
```

Data Output | Messages | Notifications

Showing rows: 1 to 3 | Page No: 1 of 1

	city character varying (100)	num_of_customers bigint	total_revenue numeric
1	Delhi	2	22200.00
2	Mumbai	2	20800.00
3	Bengaluru	2	14500.00

b) Find the month with the highest total order amount. Group by month using DATE_TRUNC or EXTRACT.

Query | Query History

```sql
SELECT
    EXTRACT(YEAR FROM order_date) AS year,
    EXTRACT(MONTH FROM order_date) AS month,
    SUM(total_amount) AS total_order_amount
FROM orders
GROUP BY
    EXTRACT(YEAR FROM order_date),
    EXTRACT(MONTH FROM order_date)
ORDER BY total_order_amount DESC
LIMIT 1;
```

Data Output | Messages | Notifications

Showing rows: 1 to 1 | Page No: 1 of 1

	year numeric	month numeric	total_order_amount numeric
1	2024	1	19500.00

c) Average order amount per customer. Only show customers with 3+ orders AND average > 500.

Query | Query History

```sql
SELECT c.id,c.name,COUNT(*) as num_of_orders,AVG(o.total_amount) as average_amount
FROM customers c
JOIN orders o
ON c.id=o.customer_id
GROUP BY c.id,c.name
HAVING COUNT(*)>=3 AND AVG(o.total_amount)>500;
```

Data Output | Messages | Notifications

Showing rows: 1 to 2 | Page No: 1 of 1

	id [PK] integer	name character varying (100)	num_of_orders bigint	average_amount numeric
1	3	Ishaan	4	4125.0000000000000000
2	1	Aarav	4	4125.0000000000000000

d) Count distinct customers who placed an order each month of 2024.

Query | Query History

```sql
SELECT
    EXTRACT(MONTH FROM order_Date) AS month,
    COUNT(DISTINCT customer_id) AS distinct_customers
FROM orders
WHERE order_date >= DATE '2024-01-01'
  AND order_date < DATE '2025-01-01'
GROUP BY EXTRACT(YEAR FROM order_date),EXTRACT(MONTH FROM order_Date)
ORDER BY month;
```

Data Output | Messages | Notifications

Showing rows: 1 to 7 | Page No: 1 of 1

	month numeric	distinct_customers bigint
1	1	4
2	2	2
3	3	2
4	4	1
5	5	1
6	6	2
7	7	1

e) Predict the result set: -- orders: (1,'Mumbai',500), (2,'Mumbai',300),(3,'Delhi',800),(4,'Delhi',400), (5,'Mumbai',700)

```
SELECT city, COUNT(*) cnt, SUM(total_amount) rev FROM orders GROUP BY city
HAVING SUM(total_amount)>1000 ORDER BY rev DESC;
```

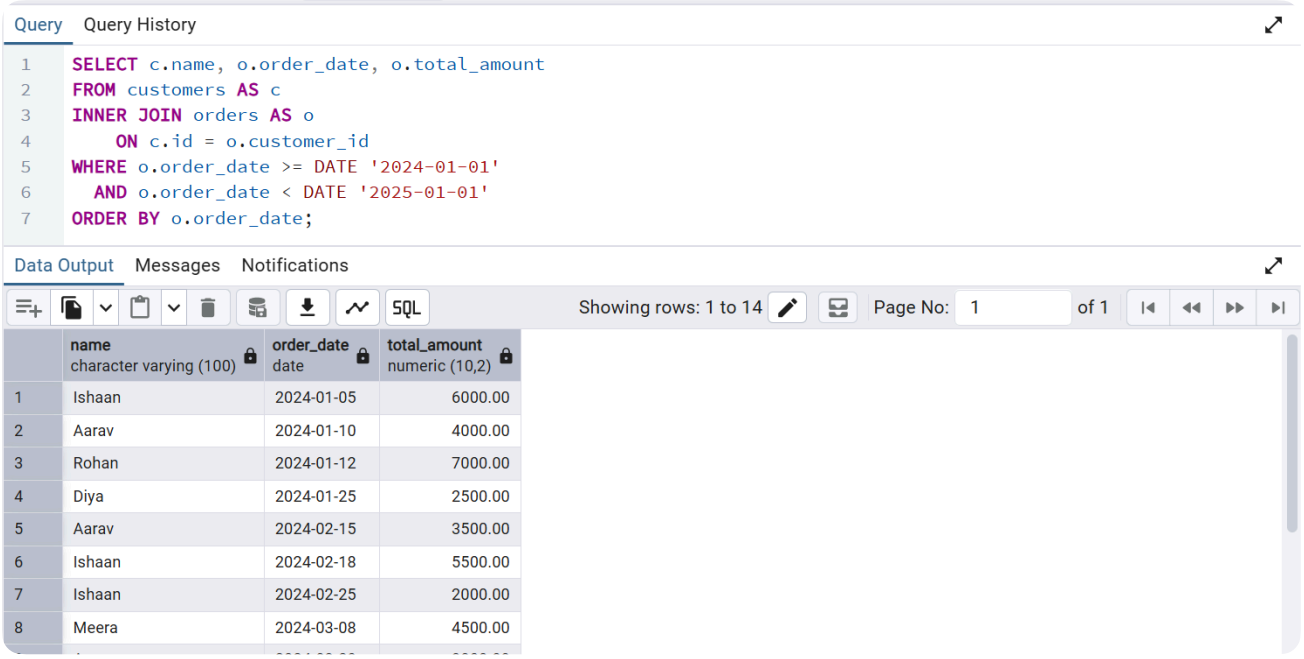
Final result set:

city	cnt	rev
Mumbai	3	1500
Delhi	2	1200

Q3) Topic: JOINs: INNER, LEFT, RIGHT & FULL OUTER

Using customers, orders, and products tables.

a) INNER JOIN: customer name, order date, total amount for all 2024 orders.



b) LEFT JOIN: ALL customers and their order count (show 0 if no orders). Explain LEFT vs INNER in a comment.

Query | Query History

```sql
SELECT c.name, COUNT(o.id) AS order_count
FROM customers AS c
LEFT JOIN orders AS o
    ON c.id = o.customer_id
GROUP BY c.id;
```

Data Output | Messages | Notifications

Showing rows: 1 to 7 | Page No: 1 of 1

	name character varying (100)	order_count bigint
1	Ishaan	4
2	Rohan	2
3	Meera	2
4	Sara	2
5	Diya	2
6	Kabir	0
7	Aarav	4

c) Find products that have NEVER appeared in any order. Use LEFT JOIN + IS NULL or NOT EXISTS -- explain your choice.

Query | Query History

```sql
SELECT p.id, p.name
FROM products AS p
WHERE NOT EXISTS (
    SELECT 1
    FROM order_items AS oi
    WHERE oi.product_id = p.id
);
```

Data Output | Messages | Notifications

Showing rows: 1 to 6 | Page No: 1 of 1

	id [PK] integer	name character varying (50)
1	11	Monitor
2	12	Keyboard
3	5	Smartwatch
4	8	Desk
5	6	Camera
6	7	Office Chair

Explanation:

- The subquery checks whether each product appears in `order_items`.
- `NOT EXISTS` returns `TRUE` when no matching record is found.
- Therefore, the query returns only products that have never been ordered.

Reason for the choice: `NOT EXISTS` expresses the requirement directly and avoids the need to check for `NULL` values after a `LEFT JOIN`. Both approaches are valid, but `NOT EXISTS` makes the intent clear.

d) Write a FULL OUTER JOIN example and explain which rows appear that would NOT appear in INNER, LEFT, or

RIGHT JOINs.

Query | Query History

```
SELECT
    c.id AS customer_id,
    c.name,
    o.id AS order_id,
    o.total_amount
FROM customers AS c
FULL OUTER JOIN orders AS o
    ON c.id = o.customer_id
ORDER BY c.id, o.id;
```

Data Output | Messages | Notifications

Showing rows: 1 to 17 | Page No: 1 of 1

	customer_id integer	name character varying (100)	order_id integer	total_amount numeric (10,2)
1	1	Aarav	1	4000.00
2	1	Aarav	2	3500.00
3	1	Aarav	3	3000.00
4	1	Aarav	15	6000.00
5	2	Diya	4	2500.00
6	2	Diya	5	1800.00
7	3	Ishaan	6	6000.00

Explanation:

- Matching customers and orders appear together.
- Customers without orders appear with `NULL` in the order columns.
- Orders without a matching customer appear with `NULL` in the customer columns.

e) Predict the result set: -- customers: (1,'Alice'),(2,'Bob'),(3,'Carol') -- orders: (101,1,500),(102,1,300)

```
SELECT c.name, COUNT(o.id) AS order_count
FROM customers c LEFT JOIN orders o
ON c.id=o.customer_id GROUP BY c.name ORDER BY c.name;
```

Final result set:

name	order_count
Alice	2
Bob	0
Carol	0

Q4) Topic: Subqueries: Scalar, Correlated, EXISTS / NOT EXISTS vs IN / NOT IN

Using the same e-commerce tables:

a) Scalar subquery in SELECT: each order alongside the overall average order amount.

Query | Query History

```
SELECT c.id AS customer_id, c.name, o.id AS order_id, o.total_amount
FROM customers AS c
FULL OUTER JOIN orders AS o
    ON c.id = o.customer_id
ORDER BY c.id, o.id;
```

Data Output | Messages | Notifications

Showing rows: 1 to 17 | Page No: 1 of 1

	customer_id integer	name character varying (100)	order_id integer	total_amount numeric (10,2)
1	1	Aarav	1	4000.00
2	1	Aarav	2	3500.00
3	1	Aarav	3	3000.00
4	1	Aarav	15	6000.00
5	2	Diya	4	2500.00
6	2	Diya	5	1800.00
7	3	Ishaan	6	6000.00
8	3	Ishaan	7	5500.00
9	3	Ishaan	8	2000.00
10	3	Ishaan	16	3000.00

b) Use EXISTS to find customers who placed an order in the last 30 days. Rewrite using IN. Explain which is more performant on large tables and why.

Query | Query History

```
SELECT c.id, c.name
FROM customers AS c
WHERE EXISTS (
    SELECT 1
    FROM orders AS o
    WHERE o.customer_id = c.id
      AND o.order_date >= DATE '2024-01-10' - INTERVAL '30 days'
      AND o.order_date <= DATE '2024-01-10'
);
```

Data Output | Messages | Notifications

Showing rows: 1 to 2 | Page No: 1 of 1

	id [PK] integer	name character varying (100)
1	1	Aarav
2	3	Ishaan

c) Use NOT EXISTS to find customers who have NEVER placed an order. Compare with NOT IN -- explain the NULL trap.

Query | Query History

```sql
SELECT c.id, c.name
FROM customers AS c
WHERE NOT EXISTS (
    SELECT 1
    FROM orders AS o
    WHERE o.customer_id = c.id
);
```

Data Output | Messages | Notifications

Showing rows: 1 to 1 | Page No: 1 of 1

	id [PK] integer	name character varying (100)
1	7	Kabir

The `NOT IN` version can fail to return expected results if the subquery contains a `NULL`.

For example, if the subquery returns `(1, 2, NULL)`, checking whether customer ID `3` is not in that set evaluates to `UNKNOWN`, not `TRUE`. SQL excludes that row in the `WHERE` clause.

`NOT EXISTS` avoids this particular NULL trap because it checks for matching rows rather than comparing against a set that might contain `NULL`.

Choice: Prefer `NOT EXISTS` for finding customers without orders, especially when the subquery column might contain `NULL`.

d) Use a derived table (subquery in FROM) to calculate per-customer totals, then select customers above the overall average.

Query | Query History

```sql
SELECT customer_totals.customer_id, customer_totals.total_spent
FROM (SELECT customer_id,
             SUM(total_amount) AS total_spent
    FROM orders
    GROUP BY customer_id
) AS customer_totals
WHERE customer_totals.total_spent > (
    SELECT AVG(total_spent)
    FROM (SELECT customer_id,
                 SUM(total_amount) AS total_spent
          FROM orders
          GROUP BY customer_id
    ) AS all_customer_totals
);
```

Data Output | Messages | Notifications

Showing rows: 1 to 3 | Page No: 1 of 1

	customer_id integer	total_spent numeric
1	3	16500.00
2	5	11000.00
3	1	16500.00

e) Predict the result set: -- orders: (1,100),(2,200),(3,300),(4,400),(5,500)

```
SELECT id, total_amount, (SELECT AVG(total_amount) FROM orders) AS avg FROM
orders WHERE total_amount > (SELECT AVG(total_amount) FROM orders);
```

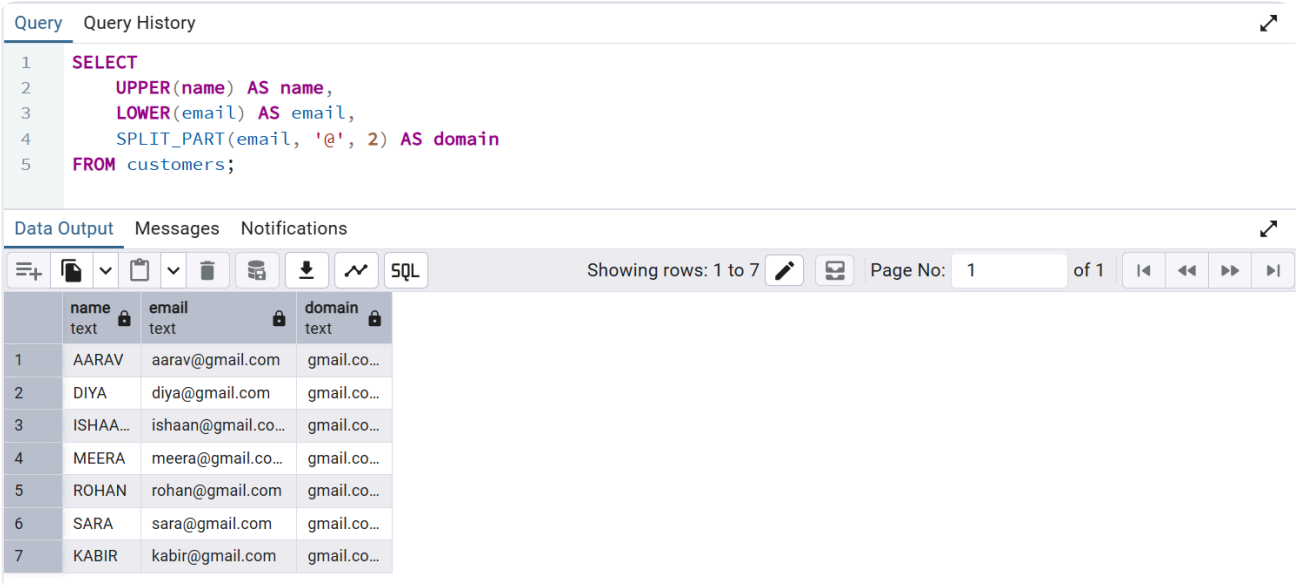
Final result set:

id	total_amount	avg
4	400	300
5	500	300

Q5) Topic: String & Date/Time Functions + CASE WHEN

Using customers (id, name, email, registration_date) and orders (order_date):

a) Show name in UPPER CASE, email in LOWER CASE, and the domain part of the email (everything after '@').



b) Format registration_date as 'DD-Month-YYYY'. Use TO_CHAR or DATE_FORMAT.Format registration_date as 'DD-Month-YYYY'. Use TO_CHAR or DATE_FORMAT.

Query | Query History

```
SELECT id, name, TO_CHAR(registration_date, 'DD-Month-YYYY') AS formatted_date
FROM customers;
```

Data Output | Messages | Notifications

Showing rows: 1 to 7 | Page No: 1 of 1

	id [PK] integer	name character varying (100)	formatted_date text
1	1	Aarav	20-October -20...
2	2	Diya	15-September-2...
3	3	Ishaan	10-August -2026
4	4	Meera	25-July -2026
5	5	Rohan	01-October -20...
6	6	Sara	18-June -2026
7	7	Kabir	28-October -20...

c) Calculate how many days ago each customer registered. Show customers registered in the last 90 days.

Query | Query History

```
SELECT id, name, CURRENT_DATE - registration_date AS days_ago
FROM customers
WHERE registration_date >= CURRENT_DATE - INTERVAL '90 days'
  AND registration_date <= CURRENT_DATE;
```

Data Output | Messages | Notifications

Showing rows: 1 to 5 | Page No: 1 of 1

	id [PK] integer	name character varying (100)	days_ago integer
1	1	Aarav	8
2	2	Diya	43
3	3	Ishaan	79
4	5	Rohan	27
5	7	Kabir	0

d) CASE WHEN label on products: 'Premium' (>1000), 'Standard' (200-1000), 'Budget' (<200). Only show products in at least one order.

Query | Query History

```
SELECT p.id, p.name, p.price,
CASE WHEN p.price > 1000 THEN 'Premium'
     WHEN p.price >= 200 THEN 'Standard'
     ELSE 'Budget'
END AS price_category
FROM products AS p
WHERE EXISTS (
    SELECT 1
    FROM order_items AS oi
    WHERE oi.product_id = p.id
);
```

Data Output | Messages | Notifications

Showing rows: 1 to 6 | Page No: 1 of 1

	id [PK] integer	name character varying (50)	price numeric (10,2)	price_category text
1	1	Phone	999.00	Standard
2	2	Laptop	1500.00	Premium
3	3	Tablet	500.00	Standard
4	4	Headphones	750.00	Standard
5	9	Notebook	50.00	Budget

e) Predict the result set: -- customer: id=1, name='john doe', email='john.doe@gmail.com'

```
SELECT UPPER(name) AS name, LOWER(email) AS email,
SPLIT_PART(email,'@',2) AS domain
FROM customers WHERE id=1;
```

Final result set:

name	email	domain
JOHN DOE	john.doe@gmail.com	gmail.com

Q6) Topic: DDL: CREATE TABLE, ALTER TABLE & Normalization (1NF to 3NF)

Un-normalized table: orders_flat(order_id, customer_name, customer_email, customer_city, product1_name, product1_price, product2_name, product2_price, salesperson_name, salesperson_region):

a) Identify every normalization violation. Label each as 1NF, 2NF, or 3NF violation.

Given:

```
orders_flat(order_id, customer_name, customer_email, customer_city,
product1_name, product1_price, product2_name, product2_price,
salesperson_name, salesperson_region)
```

1NF Violation

1. `product1_name`, `product1_price`, `product2_name`, and `product2_price` represent repeating product groups.
2. An order with more than two products would require additional columns.
3. Store each product ordered as a separate row in an `order_items` table instead.

2NF Violation

1. Assuming the original table uses a composite key such as `(order_id, product_name)`, customer and salesperson details depend only on `order_id`, not

on the entire composite key.

2. Product price and product name depend on the product identifier, not on the entire order-item key.
3. These are partial dependencies, which violate 2NF when the composite key is the table's candidate key.

3NF Violation

1. If `customer_email` determines `customer_name` and `customer_city`, then customer details depend on the customer rather than directly on the order.
2. If `salesperson_name` determines `salesperson_region`, then region depends on the salesperson rather than directly on the order.
3. These are transitive dependencies under the assumed functional dependencies.

b) Redesign into separate tables satisfying 3NF. Name each table and list columns with PKs and FKs.

Use five tables to separate customer, product, salesperson, order, and order-item information.

Table	Columns	Purpose
`customers`	`customer_id` (PK), `customer_name`, `customer_email`, `customer_city`	Customer details
`salespersons`	`salesperson_id` (PK), `salesperson_name`, `salesperson_region`	Salesperson details
`products`	`product_id` (PK), `product_name`, `price`	Product details
`orders`	`order_id` (PK), `customer_id` (FK), `salesperson_id` (FK), `order_date`	Order-level details
`order_items`	`order_id` (PK, FK), `product_id` (PK, FK), `quantity`, `unit_price`	Products belonging to each order

c) Write CREATE TABLE SQL for your 3NF design. Include PRIMARY KEY, FOREIGN KEY, NOT NULL, and CHECK constraints.

```
CREATE TABLE customers (
    customer_id INT PRIMARY KEY,
    customer_name VARCHAR(100) NOT NULL,
    customer_email VARCHAR(255) NOT NULL,
```

```
    customer_city VARCHAR(100) NOT NULL
);

CREATE TABLE salespersons (
    salesperson_id INT PRIMARY KEY,
    salesperson_name VARCHAR(100) NOT NULL,
    salesperson_region VARCHAR(100) NOT NULL
);

CREATE TABLE products (
    product_id INT PRIMARY KEY,
    product_name VARCHAR(100) NOT NULL,
    price NUMERIC(10, 2) NOT NULL
);

CREATE TABLE orders (
    order_id INT PRIMARY KEY,
    customer_id INT NOT NULL,
    salesperson_id INT NOT NULL,
    order_date DATE NOT NULL DEFAULT CURRENT_DATE,
    FOREIGN KEY (customer_id)
        REFERENCES customers(customer_id),
    FOREIGN KEY (salesperson_id)
        REFERENCES salespersons(salesperson_id)
);

CREATE TABLE order_items (
    order_id INT NOT NULL,
    product_id INT NOT NULL,
    quantity INT NOT NULL CHECK (quantity > 0),
    unit_price NUMERIC(10, 2) NOT NULL
        CHECK (unit_price > 0),
    PRIMARY KEY (order_id, product_id),
    FOREIGN KEY (order_id)
        REFERENCES orders(order_id),
    FOREIGN KEY (product_id)
        REFERENCES products(product_id)
);
```

d) Write ALTER TABLE to: add phone_number VARCHAR(15) to customers, add UNIQUE on email, add CHECK ensuring price > 0.

```
ALTER TABLE customers
```

```
ADD COLUMN phone_number VARCHAR(15);
```

```
ALTER TABLE customers
ADD CONSTRAINT uq_customers_email
UNIQUE (customer_email);
```

```
ALTER TABLE products
ADD CONSTRAINT chk_products_price
CHECK (price > 0);
```

e) Predict which INSERT statements succeed or fail:

```
CREATE TABLE emp(id INT PRIMARY KEY, name VARCHAR(50) NOT NULL, sal DECIMAL
CHECK(sal>0));

INSERT INTO emp VALUES (1,'Alice',50000); -- A
INSERT INTO emp VALUES (2,NULL,60000);   -- B
INSERT INTO emp VALUES (3,'Bob',-100);   -- C
```

Statement	Result	Reason
A	Succeeds	ID is unique, name is not NULL, and salary is positive.
B	Fails	`name` violates the `NOT NULL` constraint.
C	Fails	Salary violates `CHECK (sal > 0)`.